

4.11 Debugging Forks

On most systems, GDB has no special support for debugging programs which create additional processes using the `fork` function. When a program forks, GDB will continue to debug the parent process and the child process will run unimpeded. If you have set a breakpoint in any code which the child then executes, the child will get a `SIGTRAP` signal which (unless it catches the signal) will cause it to terminate.

However, if you want to debug the child process there is a workaround which isn't too painful. Put a call to `sleep` in the code which the child process executes after the fork. It may be useful to sleep only if a certain environment variable is set, or a certain file exists, so that the delay need not occur when you don't want to run GDB on the child. While the child is sleeping, use the `ps` program to get its process ID. Then tell GDB (a new invocation of GDB if you are also debugging the parent process) to attach to the child process (see [Attach](#)). From that point on you can debug the child process just like any other process which you attached to.

On some systems, GDB provides support for debugging programs that create additional processes using the `fork` or `vfork` functions. On GNU/Linux platforms, this feature is supported with kernel version 2.5.46 and later.

The fork debugging commands are supported in native mode and when connected to `gdbserver` in either `target remote` mode or `target extended-remote` mode.

By default, when a program forks, GDB will continue to debug the parent process and the child process will run unimpeded.

If you want to follow the child process instead of the parent process, use the command `set follow-fork-mode`.

```
set follow-fork-mode mode
```

Set the debugger response to a program call of `fork` or `vfork`. A call to `fork` or `vfork` creates a new process. The *mode* argument can be:

`parent`

The original process is debugged after a fork. The child process runs unimpeded. This is the default.

`child`

The new process is debugged after a fork. The parent process runs unimpeded.

```
show follow-fork-mode
```

Display the current debugger response to a `fork` or `vfork` call.

On Linux, if you want to debug both the parent and child processes, use the command `set detach-on-fork`.

```
set detach-on-fork mode
```

Tells `gdb` whether to detach one of the processes after a fork, or retain debugger control over them both.

`on`

The child process (or parent process, depending on the value of `follow-fork-mode`) will be detached and allowed to run independently. This is the default.

off

Both processes will be held under the control of GDB. One process (child or parent, depending on the value of `follow-fork-mode`) is debugged as usual, while the other is held suspended.

`show detach-on-fork`

Show whether `detach-on-fork` mode is on/off.

If you choose to set ‘`detach-on-fork`’ mode off, then GDB will retain control of all forked processes (including nested forks). You can list the forked processes under the control of GDB by using the `info inferiors` command, and switch from one fork to another by using the `inferior` command (see [Debugging Multiple Inferiors Connections and Programs](#)).

To quit debugging one of the forked processes, you can either detach from it by using the `detach inferior` command (allowing it to run independently), or kill it using the `kill inferior` command. See [Debugging Multiple Inferiors Connections and Programs](#).

If you ask to debug a child process and a `vfork` is followed by an `exec`, GDB executes the new target up to the first breakpoint in the new target. If you have a breakpoint set on `main` in your original program, the breakpoint will also be set on the child process’s `main`.

On some systems, when a child process is spawned by `vfork`, you cannot debug the child or parent until an `exec` call completes.

If you issue a `run` command to GDB after an `exec` call executes, the new target restarts. To restart the parent process, use the `file` command with the parent executable name as its argument. By default, after an `exec` call executes, GDB discards the symbols of the previous executable image. You can change this behavior with the `set follow-exec-mode` command.

`set follow-exec-mode mode`

Set debugger response to a program call of `exec`. An `exec` call replaces the program image of a process.

`follow-exec-mode` can be:

`new`

GDB creates a new inferior and rebinds the process to this new inferior. The program the process was running before the `exec` call can be restarted afterwards by restarting the original inferior.

For example:

```
(gdb) info inferiors
(gdb) info inferior
  Id  Description  Executable
* 1   <null>      prog1
(gdb) run
process 12020 is executing new program: prog2
Program exited normally.
(gdb) info inferiors
  Id  Description  Executable
  1   <null>      prog1
* 2   <null>      prog2
```

`same`

GDB keeps the process bound to the same inferior. The new executable image replaces the previous executable loaded in the inferior. Restarting the inferior after the `exec` call, with e.g., the `run` command, restarts the executable the process was running after the `exec` call. This is the default mode.

For example:

```
(gdb) info inferiors
  Id   Description  Executable
* 1    <null>       prog1
(gdb) run
process 12020 is executing new program: prog2
Program exited normally.
(gdb) info inferiors
  Id   Description  Executable
* 1    <null>       prog2
```

follow-exec-mode is supported in native mode and target extended-remote mode.

You can use the catch command to make GDB stop whenever a fork, vfork, or exec call is made. See [Setting Catchpoints](#).

Next: [Checkpoint/Restart](#), Previous: [Threads](#), Up: [Running](#) [[Contents](#)][[Index](#)]